

Write Operation with PrintWriter

PrintWriter

In Java file operations, we've seen that `FileWriter` and `BufferedWriter` allow us to write data to files using the `write()` method. However, these classes have a limitation - they always convert the data to characters. For example, if you try to write the number `65`, it will be stored as the character `'A'` (ASCII value 65) rather than as the digits `"65"`.

`PrintWriter` is a specialized class in Java that solves this problem by allowing us to write different data types to a file while preserving their actual format. This means numbers will be written as numbers, booleans as booleans, and so on.

- **Data Type Preservation:** Writes various data types in their original format
- **Easy to Use:** Provides simple methods like `println()` for different data types
- **Convenience:** Handles formatting and line separators automatically

Example:

```
import java.io.*;

public class LaunchIO {
    public static void main(String[] args) throws IOException {
        // Define the file path where we want to write data
        String filePath = "D:\\telusko\\InputOutput\\java.txt";

        // Declare FileWriter and PrintWriter objects outside try block for scope in finally
        FileWriter fileWriter = null;
        PrintWriter printWriter = null;

        try {
            // Create a File object representing the physical file
            File file1 = new File(filePath);

            // Create a FileWriter to write characters to the file
            fileWriter = new FileWriter(file1);

            // Create a PrintWriter that wraps the FileWriter
            // PrintWriter adds the ability to write various data types as they are
            printWriter = new PrintWriter(fileWriter);

            // Using write() method still converts numbers to their character equivalents
            // This will write 'A' (the character with ASCII value 65)
            printWriter.write(65);
            printWriter.write("\n"); // Add a new line

            // Using println() methods to write different data types as they are
            printWriter.println("JAVA"); // Writes the string "JAVA"

            printWriter.println('A');    // Writes the character 'A'

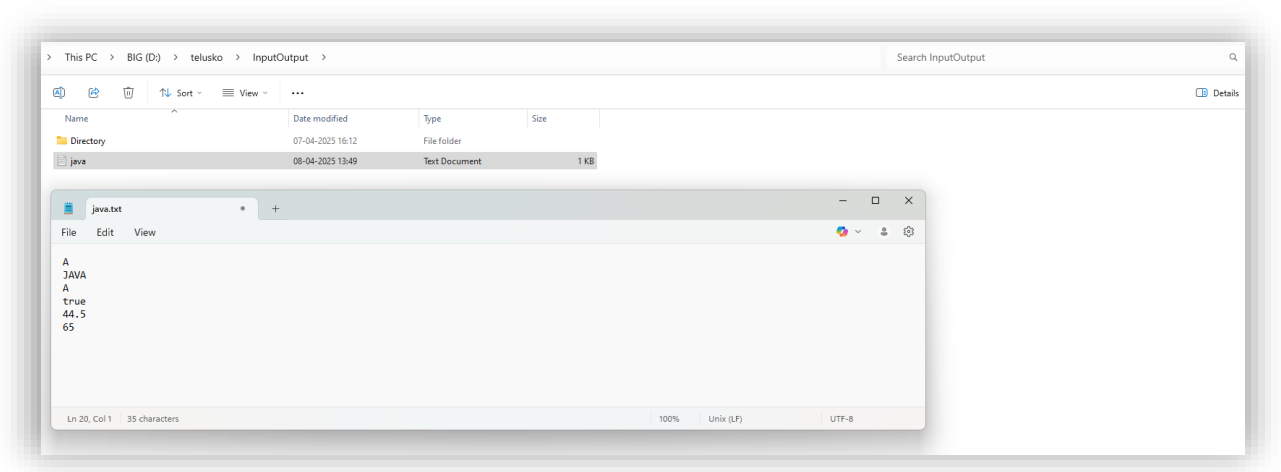
            printWriter.println(true);   // Writes the boolean value "true"

            printWriter.println(44.5);   // Writes the decimal value "44.5"

            printWriter.println(65);     // Writes the integer "65" (not 'A')

        } catch (Exception e) {
            System.out.println("Some problem");
        } finally {
            // Always close resources in finally block to ensure they're closed
            // even if an exception occurs
            printWriter.close(); // This also closes the underlying FileWriter
        }
    }
}
```

Output:



👉 Explanation

➤ File Path Setup:

- We start by defining where we want to create or write to our file.

➤ Resource Declaration:

- `FileWriter` and `PrintWriter` are declared outside the try block so they can be accessed in the finally block.

➤ File Creation and Setup:

- We create a `File` object representing the physical file.
- The `FileWriter` is created to handle basic character writing.
- The `PrintWriter` wraps around the `FileWriter` to add more functionality.

➤ Two Ways of Writing:

- `printWriter.write(65)` - This still uses the character conversion, writing 'A'.
- `printWriter.println()` methods - These preserve the data type format.

➤ Multiple Data Types Demo:

- We write a string: "JAVA"
- A character: 'A'
- A boolean: true
- A decimal: 44.5
- An integer: 65 (stored as "65", not as 'A')

➤ Resource Management:

- The finally block ensures resources are closed properly, even if errors occur.

👉 Key Differences Between `write()` and `println()`

- `write()` method: Always converts to character representation

Example: `write(65)` writes the character 'A'

- `println()` methods: Preserve the actual format of data

Example: `println(65)` writes the digits "65"